ACC : le bilan électrique de deux maisons

Guide public de mise en service et de vérification
APsystems • ESP32 • Shelly • Home Assistant et domotique

Ce projet vise le pilotage de la production dans une **autoconsommation collective (ACC), notamment entre deux maisons**. L'objectif est de faire lire à l'ECU APsystems la somme des puissances réseau mesurées dans les deux maisons, afin que sa fonction de bridage zéro export utilise leur bilan cumulé.

Bilan transmis à l'ECU = puissance réseau maison 1 + puissance réseau maison 2. Les valeurs sont signées : soutirage positif, export négatif. Exemple théorique : une maison exporte 1 000 W et l'autre soutire 1 000 W ; leur bilan cumulé vaut 0 W. Le zéro visé porte donc sur l'ensemble des deux maisons.

Ce qui fonctionne et ce qui reste à faire

La liaison **un Shelly → ESP32 → ECU → micro-onduleurs** est validée. Un essai temporaire à +1 000 W a confirmé la prise en compte de la mesure et l'action sur la production.
L'intégration du deuxième Shelly reste à réaliser ; elle ne présente a priori pas de difficulté technique particulière. L'addition des deux mesures et le bridage sur ce bilan devront ensuite être validés en conditions réelles.

Matériel testé : **ECU-C-Z C1.2.27A, Shelly Pro EM 50 et ESP32 v0.8.8**. Le profil est monophasé, avec la puissance réseau sur la phase A et dans le total émulé. Le triphasé nécessite une adaptation. Prévoir aussi un ordinateur, un câble USB de données et la sauvegarde des réglages initiaux de l'ECU.

Parcours	À effectuer
Mise en service	Pages 2 à 6 : réunir les fichiers, adapter les paramètres, compiler, sélectionner et préparer la lecture directe.
Vérification	Page 7 : prouver l'origine de la mesure, bridage désactivé. Page 8 : prouver la réaction de production, bridage actif.
Analyse du firmware	Pages 9 à 11 : récupérer et analyser le code si nécessaire. À réaliser avant la sélection si la version ECU diffère.
Domotique et exploitation	Page 12 : Home Assistant, MQTT et autres adaptations. Page 13 : retour aux tores, incidents et documentation.

Ce qui dépend du poste

Le parcours réalisé utilise Windows et Ubuntu sous WSL. macOS et Linux demandent une adaptation des outils ; Android et iPhone servent à consulter les interfaces et à régler l'ECU. Home Assistant et les autres solutions domotiques peuvent agréger les mesures après développement et validation de leur interface.

Ce PDF décrit la méthode ; les programmes sont à obtenir séparément. Les données du site sont remplacées par des exemples théoriques. Le montage à deux maisons est l'objectif, tandis que les essais déjà validés portent sur un seul Shelly.

2 Réunir les fichiers et relever les paramètres

Programmes nécessaires, à obtenir séparément

Composant	Usage
Programme ESP32 v0.8.8	ESP32_v088.ino avec WifiLocal.h, Test1000.h, ShellyWebSocket.h et CooperativeWebServer.h.
Lecteur de signature +1 000 W	Test avec zéro export désactivé : relever.py, lecture.sh et le lanceur Windows.
Lecteur de bridage +1 000 W	Test avec zéro export actif : relever.py, bridage.py, onduleurs.py, lecture.sh et le lanceur Windows.
Collecte v3.3 et analyse REA	Récupération du code et analyse hors ligne, si la version ECU ou le protocole doit être vérifié.

Les sources du prototype contiennent des paramètres propres à leur installation. Utiliser une copie nettoyée et adaptée. Ce PDF ne distribue ni code de firmware constructeur, ni identifiants, ni archives privées.

Extraire les deux kits de test dans des dossiers distincts. Ils possèdent chacun un Relever_les_watts.cmd et un relever.py différents. Ne pas mélanger leurs fichiers.

Fiche à compléter pour la nouvelle installation

Donnée	Comment la relever
IP ECU, IP ESP32, IP Shelly	Liste des appareils du routeur ; adresse de l’ESP32 également affichée dans le moniteur série.
UID et version ECU	Informations EMA Manager ou lecture d’identité de l’ESP32. Noter la chaîne complète du modèle et du firmware.
Identifiant du Shelly	Champ sys.mac du JSON Shelly.GetStatus, sans séparateurs dans la comparaison du prototype.
Canal réseau Shelly	Identifier le canal qui mesure le point d’échange maison/réseau : em1:0 ou em1:1 dans ce modèle.
ID compteur ESP32	Champ meter_id de http://IP_ESP/status après le premier démarrage. Le relever exactement.
Configuration initiale ECU	Pour Grid et PV : type, activation, phase PH, ratioA/B/C et reverseA/B/C. Conserver la réponse complète.
Wi-Fi et carte	SSID, mot de passe local, modèle ESP32, port série et version du cœur Arduino.

IP_ECU, IP_ESP, IP_SHELLY, UID_ECU et ID_COMPTEUR sont des paramètres à remplacer localement. Les adresses réseau et identifiants réels ne figurent pas dans cette édition publique.

3 Préparer le réseau et démarrer la carte

1. Relier l’ECU, le Shelly, l’ESP32 et le poste au réseau local qui permet leurs échanges. Pendant les relevés, utiliser ce réseau sur le téléphone ; une connexion au seul point d’accès de l’ECU ne démontre pas l’accès au Shelly. Stabiliser les adresses dans le routeur si son service DHCP les change.

2. Ouvrir cette adresse dans un navigateur, en remplaçant IP_SHELLY :

```
http://IP_SHELLY/rpc/Shelly.GetStatus
```

Repérer sys.mac, le canal em1:N et act_power. Dans la convention du montage validé, une puissance positive indique du soutirage et une puissance négative de l’export. Comparer au sens indiqué dans l’application Shelly. Le programme actuel attend un accès local sans authentification ; une réponse 401 exige une prise en charge de cette authentification dans le client.

3. Installer Arduino IDE sur Windows, macOS ou Linux [S1]. Dans les préférences, ajouter l’URL de cartes Espressif [S2], puis installer « esp32 by Espressif Systems » dans le gestionnaire de cartes. La procédure de référence utilise la version 3.3.11.

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

4. Ouvrir ESP32_v088/ESP32_v088.ino dans le dossier du programme. Garder les quatre fichiers .h voisins. Dans WifiLocal.h, remplir les deux chaînes :

```
constexpr char WIFI_SSID[] = "VOTRE_SSID";
constexpr char WIFI_PASSWORD[] = "VOTRE_MOT_DE_PASSE";
```

Sous Windows, Preparer_ESP32.cmd peut récupérer ces deux valeurs dans un ancien sketch v0.8.7 déjà configuré. Sur une nouvelle installation, la saisie manuelle convient. Ces valeurs restent dans la copie locale de travail ; le fichier partagé doit contenir des paramètres vides ou fictifs.

5. Pour la carte classique de référence, choisir ESP32 Dev Module et son port USB. Cliquer sur Vérifier puis Téléverser. Ouvrir le moniteur série à 115200 bauds. Sur une autre famille ESP32, choisir la carte correspondante et revalider la compatibilité du programme.

Résultat attendu : Le moniteur indique v0.8.8 et « Autotest local : OK ». La page http://IP_ESP/ s’ouvre. Ce premier démarrage sert au repérage des identifiants ; effectuer les adaptations des pages 4 et 5 avant de sélectionner le compteur dans l’ECU.

4 Adapter les identités et la mesure Shelly

Dans Arduino IDE, rechercher les noms ci-dessous dans ESP32_v088.ino. Modifier leurs valeurs pour le site cible. Garder une copie du programme original et noter les changements dans un fichier de paramètres du site.

Emplacement dans le sketch	Valeur à fournir
ecuTargetText	IP de l'ECU. La page permet également de saisir cette adresse.
EXPECTED_ECU_UID	UID exact de l'ECU cible, sur 12 chiffres.
EXPECTED_ECU_VERSION	Chaîne complète réellement lue. Référence : ECU-C-Z_C1.2.27A. Une autre version nécessite la vérification du protocole avant utilisation.
EXPECTED_METER_ID	Copie exacte du champ meter_id de la carte cible, lu sur /status.
SHELLY_IP_TEXT	IP du Shelly mesurant le réseau.
SHELLY_MAC	Identité sys.mac du Shelly cible.
SHELLY_CHANNEL et decodeShellyStatus	Canal choisi et clés JSON correspondantes : em1:N et em1data:N. Modifier les trois éléments ensemble.

Relever les identifiants sans changer la source réseau

1. Ouvrir http://IP_ESP/status et relever meter_id, host et instance. Utiliser meter_id tel qu'il est généré par le sketch ; ne pas le reconstruire à partir d'une MAC affichée autrement par le routeur.

2. Dans la page de l'ESP32, saisir l'adresse de l'ECU et demander ses informations. Relever ecu_uid et ecu_version_model dans /status. La lecture d'identité peut afficher les valeurs même si elles ne correspondent pas encore aux constantes attendues.

3. Reporter ces valeurs dans le sketch, puis compiler et téléverser à nouveau. L'identité de la carte reste la même. Vérifier self_test_ok et ecu_identity_matches_expected à true après la nouvelle lecture d'identité.

Conserver la règle de nommage qui a fonctionné

Pour le profil étudié, le nom d'hôte est **M01 + meter_id + .local**, avec la casse exacte. Le programme le génère déjà. Exemple de structure : M01LABxxxxxxxxxxx.local. Le préfixe M01 permet à C1.2.27A de reconnaître le compteur ; l'ID qui suit doit être identique à celui transmis dans sa configuration.

Le port compteur 9527 est un choix du prototype. Le conserver simplifie la reprise. S'il est modifié, adapter ensemble le serveur, l'annonce et les champs de configuration et de validation ; changer une seule constante ne suffit pas.

5 Adapter les tores puis sélectionner le compteur

Préparer un retour exact à la configuration initiale

1. Avec les identités adaptées, demander une fois le relevé du choix de compteur dans la page de l’ESP32 et conserver /diagnostic.json. Cette commande 0072 n’est pas un outil de surveillance continue : le programme signale qu’elle peut redémarrer l’ECU. L’utiliser avant les captures et respecter les attentes affichées.

2. Comparer la configuration réellement relue pour meterGrid et meterPV aux attentes de switchCtRoleMatches(). Contrôler meterType, meterEnable, PH, ratioA/B/C et reverseA/B/C. Les valeurs doivent provenir de la configuration initiale du boîtier cible.

3. Adapter **CT_GRID_BACKUP** à la sauvegarde réelle du compteur réseau, sans renvoyer les champs d’état status. Adapter aussi **switchCtRoleMatches()** aux réglages réels de Grid et de PV. Si leurs réglages diffèrent, le contrôle doit les distinguer par rôle. Modifier le seul texte de sauvegarde ne suffit pas.

Cette partie demande une adaptation du code si le site diffère. Ne pas déduire le champ PH du seul nombre de phases de la maison : recopier la valeur réellement relue dans la configuration. Conserver les réglages PV et la sauvegarde de retour propres au destinataire.

4. Compiler et téléverser la version adaptée si nécessaire. Garder l’effacement complet de la flash désactivé pour une mise à jour de la même carte sur le même site, afin de conserver son journal. Une carte réutilisée depuis un autre site demande une vérification de cet état mémorisé.

Effectuer la sélection depuis la page de l’ESP32

5. Désactiver le zéro export pour la première preuve. Démarrer la lecture Shelly ; attendre une mesure valide et vérifier que les watts correspondent au canal réseau. Contrôler aussi mdns_ready et l’autotest.

6. Cliquer une seule fois sur **Sélectionner l’ESP32 pour le réseau**. L’identité, les réglages et la fraîcheur de mesure sont contrôlés avant l’écriture. Une minute sépare les commandes ; prévoir environ deux minutes et attendre le résultat affiché.

Résultat attendu : La réponse de l’ECU confirme le compteur ESP32 côté réseau. Le compteur de production PV reste sur ses réglages de tores. Dans /status, les acquisitions et réponses compteur progressent sur plusieurs échanges.

Retour possible : « Retour aux tores réseau » demande la restauration de CT_GRID_BACKUP et attend la confirmation ECU. Fermer une page ou arrêter le répondeur ne restaure pas la sélection dans l’ECU.

6 Préparer les lecteurs de mesure directe

Les lecteurs comparent trois niveaux : le Shelly réel, les watts envoyés par l’ESP32 et les watts enregistrés par l’ECU. Ils évitent d’attendre le cycle d’affichage EMA. Préparer séparément le lecteur du test sans bridage et celui du test avec bridage.

1. Dans **chacun des deux fichiers relever.py**, adapter les constantes placées en tête de fichier :

```
ECU = "IP_ECU"
ESP = "IP_ESP"
SHELLY = "IP_SHELLY"
UID = "UID_ECU"
MODEL = "CHAINE_MODELE_FIRMWARE"
METER = "ID_COMPTEUR_ESP"
```

2. Dans la fonction **shelly_read()** des deux lecteurs, adapter la MAC attendue, la clé "em1:1" et le contrôle id != 1 au canal réel. Ces valeurs ne proviennent pas automatiquement des constantes du sketch : les deux lecteurs doivent être adaptés séparément.

3. Conserver le fichier lecture.sh propre à chaque kit. TEMPLATE_HASH vérifie ce fichier exact. Ne pas remplacer le script d’un kit par celui de l’autre. Si le collecteur lui-même doit évoluer, revoir son code et ses contrôles avant de produire une nouvelle empreinte de référence.

Poste	Démarrage depuis le bon dossier
Windows	Installer Python 3.8 ou plus, puis double-cliquer sur Relever_les_watts.cmd. Le lanceur utilise Python Windows, ou Ubuntu sous WSL en repli.
macOS ou Linux	Installer Python 3.8 ou plus, ouvrir le terminal dans le dossier du kit et lancer python3 -B -u relever.py. Variante à vérifier sur le poste cible.
Android ou iPhone	Consulter la page ESP32 et les applications. Le lecteur Python de cette procédure s’exécute sur un ordinateur.

Dans le kit de bridage, relever.py, bridage.py, onduleurs.py et lecture.sh doivent rester ensemble. Dans le premier kit, relever.py et lecture.sh suffisent au lecteur ; conserver aussi le dossier du sketch pour la préparation.

Ce que fait le lecteur sur l’ECU

Il vérifie l’identité et la disponibilité de assist, transmet un collecteur temporaire une seule fois, puis récupère les lectures. Le cache réseau est /tmp/gridQuickPower.conf ; la production vient de meterPV.ALL dans /tmp/meterDataTemp.conf. Les dates de modification repèrent les écritures. Une nouvelle écriture seule ne prouve pas une nouvelle mesure : la recouper avec l’âge de la source et la valeur envoyée par l’ESP32.

Résultat attendu : Le lecteur reconnaît les trois appareils. Une valeur ancienne reste identifiée comme telle. Un affichage 200 HTTP ou une page qui s’ouvre ne suffit pas à prouver une nouvelle acquisition.

7 Prouver que la mesure vient de l’ESP32

Kit : lecteur de signature +1 000 W
État de départ : ESP32 sélectionné, Shelly valide et zéro export désactivé. Le lanceur exige FUNC égal à la chaîne "0" dans la configuration ECU. Cet essai peut se faire sans soleil.

1. Terminer les opérations d’extraction ou de maintenance sur l’ECU. Extraire une copie neuve du kit et y reporter les paramètres du site validés page 6. Garder les réglages ECU inchangés pendant la capture.

2. Lancer Relever_les_watts.cmd sous Windows, ou python3 -B -u relever.py depuis le dossier sous macOS ou Linux. Laisser la fenêtre ouverte environ quatre à cinq minutes.

3. Le lecteur attend deux écritures ECU nouvelles et concordantes, puis demande une seule impulsion de +1 000 W. L’ESP32 l’applique pendant 90 secondes au maximum aux puissances actives de phase A et du total. Le Shelly et ses compteurs d’énergie conservent leurs mesures réelles.

4. Observer les trois périodes : avant, pendant et après. Vérifier que la mesure ECU suit la valeur envoyée par l’ESP32, avec le même décalage temporaire.

Grandeur	Exemple théorique
Shelly réel	600,5 W
ESP32 pendant le test	1 600,5 W
Cache ECU attendu	1 600 W

Le cache ECU retire les décimales par troncature vers zéro : -494,4 W devient -494 W. Une différence inférieure à un watt n’est donc pas un défaut de transmission.

Résultat attendu : La signature +1 000 W apparaît dans de nouvelles écritures ECU, puis disparaît au terme du test. Exiger plusieurs écritures concordantes dans chacune des trois périodes.

5. Conserver **releve_rapide_ECU.json**, créé à côté du lanceur. Si le résultat est indéterminé, examiner les âges, l’état de sélection, la validité Shelly et les concordances avant de conclure.

En cas d’interruption

L’ESP32 gère lui-même la limite de 90 secondes. Sa page propose « Arrêter le décalage ». Relancer dans le même dossier reprend la session existante tant que la trace temporaire reste disponible, environ quinze minutes. Pour un nouvel essai distinct, repartir d’une nouvelle extraction après la fin de la session précédente.

8 Vérifier le bridage réel avec du soleil

Kit : lecteur de bridage +1 000 W
État de départ : test de signature validé, ESP32 sélectionné, zéro export actif avec une limite de 0 W.

1. Choisir une période de soleil stable avec au moins 1 000 W de production supplémentaire disponible au-delà de la consommation de la maison. Une injection naturelle nettement supérieure à 1 000 W, observée juste avant d'activer le bridage, est un repère utile. Garder les gros appareils dans le même état pendant l'essai.

2. Activer la fonction zéro export à 0 W dans EMA Manager. Attendre que le Shelly réel se stabilise près de zéro. Garder le bridage actif pendant toute la capture. En cas de faible soleil, différer cet essai : une production trop basse ne permet pas de conclure.

3. Depuis le réseau local, lancer le lecteur du kit de bridage dans un dossier neuf, avec les paramètres de la page 6. Laisser la fenêtre ouverte environ cinq minutes. Le lecteur contrôle la consigne, les mesures de départ et une production récente avant la demande de décalage.

4. Pendant +1 000 W, attendre une hausse de production. L'ECU doit revenir vers zéro avec sa mesure décalée, tandis que le Shelly réel indique environ -1 000 W, donc de l'export. Après les 90 secondes, attendre le retour vers le fonctionnement initial.

Phase stabilisée	Réseau réel	ECU réseau	Production PV
Avant	≈ 0 W	≈ 0 W	P
Avec +1 000 W	≈ −1 000 W	≈ 0 W	P + 1 000 W
Après retour	≈ 0 W	≈ 0 W	P

Tableau théorique, sous soleil et consommation stables. P désigne la production de départ. L'essai réalisé a confirmé cet effet à environ 1 kW ; il ne garantit pas une régulation exacte au watt près.

Résultat attendu : L'export réel et la production varient de façon cohérente avec l'impulsion connue, puis reviennent après son retrait. Conserver **releve_bridage_ECU.json** et l'observation indépendante Shelly.

Dans l'essai réalisé, une confirmation ESP32 perdue en fin d'impulsion a laissé le verdict automatique « indéterminé ». Les courbes et la mesure Shelly ont néanmoins montré l'effet sur la production totale. Il faut examiner ces preuves ensemble. Cet essai ne démontre pas le délai individuel de chaque micro-onduleur ; des oscillations autour de zéro restent possibles.

9 Récupérer le code installé si nécessaire

Cette étape sert à vérifier le protocole lorsque la version ECU diffère ou à reproduire l’analyse. Le kit récupère du code applicatif, ses bibliothèques et son environnement ; il ne réalise pas une image complète de toute la mémoire flash. La procédure ci-dessous correspond au service local étudié sur l’ECU-C.

1. Sous Windows, extraire le kit de récupération v3.3. Garder Recuperer_ECU.cmd, Recuperer_ECU.ps1, Reprendre_telechargement.cmd, le paquet .tar.bz2 et le dossier sources ensemble.

2. Dans Recuperer_ECU.ps1, adapter **$Base** à http://IP_ECU et le texte d’affichage de l’adresse cible. Conserver le paquet d’origine et les empreintes de vérification. Vérifier que la page locale /index.php/hidden/upload correspond bien au service étudié avant toute collecte sur une autre version.

3. Hors mise à jour ou autre opération de maintenance, lancer Recuperer_ECU.cmd une seule fois. Le lanceur vérifie le paquet, le code de compression, la page web et l’absence de assist dans /home/update_from_app/update_localweb/.

4. Attendre la préparation puis le téléchargement dans le dossier retour_ECU_v3_3_… affiché. Le mécanisme utilise /index.php/management/exec_upgrade_ecu_app pour lancer un script de copie temporaire. Il collecte notamment main.exe, meterManager, les bibliothèques, le code web et les scripts de démarrage.

5. Vérifier la présence de ecu_etendu_v3_3.tar.gz et des rapports locaux. Si seul le transfert est interrompu, utiliser Reprendre_telechargement.cmd pendant la disponibilité de la copie, environ quinze minutes. Ce mode de reprise ne renvoie pas le script.

Contrôles à conserver

Le lanceur vérifie le transfert, la compression et la structure de l’archive. Conserver SHA256SUMS.txt, copied_paths.txt, les inventaires, les exclusions et les fichiers de version avant/après. Dans la collecte réalisée, les empreintes du manifeste ont concordé. Appliquer cette même vérification à toute nouvelle extraction.

```
Get-FileHash .\ecu_etendu_v3_3.tar.gz -Algorithm SHA256
```

Cette commande PowerShell calcule l’empreinte globale à archiver. Sous Linux : sha256sum ; sous macOS : shasum -a 256. Le nombre de fichiers et l’empreinte d’une nouvelle extraction peuvent différer ; contrôler son manifeste et son contenu, sans lui imposer ceux d’un autre boîtier.

Mac et Linux : l’analyse de l’archive est portable. Le collecteur v3.3 livré, lui, dépend de PowerShell Windows et d’un composant C#. Utiliser Windows pour cette collecte ou porter et revalider cette couche ; renommer le lanceur .cmd ne suffit pas. Si assist est occupé, conserver le refus et attendre la fin de l’opération en cours.

10 Installer REA et son moteur d’analyse

Cette page concerne le poste d’analyse. L’ESP32 fonctionne sans que REA reste ouvert. Les versions suivantes sont celles du parcours réalisé : Ghidra 12.1.4, JDK 21, Node 26.13.0, REA 6.8.0 et Codex CLI 0.166.0. [S3, S4]

Windows et Ubuntu sous WSL

Dans PowerShell administrateur, installer WSL puis redémarrer si demandé. Installer Ubuntu si la liste ne contient aucune distribution :

```
wsl --install
wsl --list --verbose
wsl --install -d Ubuntu
```

Dans Ubuntu, créer le compte Linux puis préparer Java et les utilitaires :

```
sudo apt update
sudo apt install openjdk-21-jdk curl unzip git python3
java -version
javac -version
```

Installer nvm et Node dans Ubuntu, puis REA et Codex dans le même environnement :

```
curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash
source ~/.bashrc
nvm install 26.13.0
nvm alias default 26.13.0
npm install --global rea-agents@6.8.0
npm install --global @openai/codex@0.166.0
command -v node npm rea codex
```

Télécharger Ghidra 12.1.4 depuis ses releases officielles [S5] et décompresser le ZIP dans ~/outils. Adapter les chemins, puis lancer :

```
export GHIDRA_INSTALL_DIR="$HOME/outils/ghidra_12.1.4_PUBLIC"
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export REA_ANALYSIS_PROVIDER=ghidra
"$GHIDRA_INSTALL_DIR/support/analyzeHeadless"
rea setup --client codex
codex login
```

Appliquer l’enregistrement REA proposé, puis relancer Codex. Lui demander de lire reverse-engineer-anything et d’appeler rea.binary_session({}). Le résultat doit confirmer Ghidra, Java et le décompilateur natif disponibles.

Mac ou Linux natif

Installer Java, Node et Ghidra pour le système et l’architecture du poste, puis reprendre les commandes npm et rea setup. Adapter JAVA_HOME et GHIDRA_INSTALL_DIR. Sur Mac Intel ou Apple Silicon, suivre la section du décompilateur natif correspondante dans [S3]. La version de Java doit correspondre à la version de Ghidra choisie ; ne pas appliquer automatiquement JDK 21 à une autre génération.

11 Analyser la bonne version et vérifier le protocole

Reprendre une extraction vérifiée

1. Extraire le kit d’analyse REA correspondant à la collecte. Sous Windows, lancer 1_Analyser_avec_REA.cmd et sélectionner ecu_etendu_v3_3.tar.gz. L’analyse travaille sur le PC, sans contacter les appareils.

2. Conserver le dossier de travail sous ~/REA-work/ECU-C et le retour_REA_v33_….zip créé à côté du lanceur, même s’il est partiel. Le résultat doit associer chaque conclusion au fichier analysé, à son empreinte et aux preuves disponibles.

Limite exacte du lanceur : preparer_rea.py vérifie ARCHIVE_SHA, les nombres de fichiers et d’entrées du manifeste, ainsi que la version attendue. Une nouvelle archive exige une adaptation explicite de ces validations et de la mission. Ne pas remplacer seulement l’empreinte pour faire accepter un contenu différent.

Examiner une nouvelle extraction

Contrôler d’abord son inventaire et son manifeste, puis ouvrir les exécutables dans REA/Ghidra. L’image de mise à jour C1.2.26 ne remplace pas le code C1.2.27A réellement récupéré. Les copies des exécutables actifs servent à vérifier que les fichiers étudiés sont ceux utilisés par l’ECU.

Mission à confier à l’analyse : identifier la reconnaissance mDNS et la dérivation de l’ID compteur ; suivre les acquisitions dans main.exe et meterManager ; vérifier le décodage de libmetertools.so ; retrouver les caches réseau/PV et la fonction de régulation. Demander les fonctions, adresses, empreintes et limites de preuve, sans lancer les exécutables ARM sur le poste.

Point vérifié dans le cas réussi	Valeur ou comportement
Nom du compteur	Après retrait de .local : préfixe M01, puis ID exact du compteur.
Demande RTU claire	01 04 00 01 00 58 A0 30 : unité 1, fonction 04, 88 registres dès 1.
Réponse	181 octets clairs ; 44 flottants 32 bits big endian. Puissance A à l’index 9 et total à l’index 18.
Transport chiffré	Réponse AES-128-CBC avec bourrage PKCS#7, soit 192 octets transmis.
Mesure interne	gridQuickPower.conf reçoit la puissance signée tronquée vers zéro. meterDataTemp.conf expose aussi la production PV.

La correction décisive a été le nom M01 suivi du bon ID. Un débordement IPC a été identifié dans le code, mais son rôle dans le blocage reste non confirmé ; la fragmentation TCP n’est pas démontrée. Compléter l’analyse par les essais des pages 7 et 8 pour prouver la réception et la réaction physique.

12 Adapter à Home Assistant et à la domotique

Pour l'**ACC à deux maisons**, la cible est la somme de leurs puissances réseau signées. Elle peut être calculée dans l'ESP32 ou dans une couche domotique adaptée. L'ECU conserve le pilotage du bridage. La liaison avec un Shelly est validée ; l'agrégation de deux Shelly et les interfaces domotiques restent à développer et à tester.

Usage	Ce qu'il faut prévoir
Suivi en parallèle	Home Assistant peut recevoir les mesures Shelly avec son intégration officielle [H1]. Le trajet direct vers l'ESP32 reste en place.
Home Assistant comme source	Un adaptateur récupère l'entité de puissance par API REST/WebSocket [H2], ou reçoit une publication MQTT [H3], puis alimente le modèle de mesure de l'ESP32.
Autre domotique	Un système pouvant lire les compteurs et émettre un flux local exploitable peut remplir le même rôle. Exemples de cibles à étudier : Node-RED, Jeedom ou Domoticz.
Deuxième Shelly / ACC	Intégration restant à réaliser, sans difficulté technique particulière identifiée a priori. Additionner les bilans des deux maisons, puis valider le bridage sur cette somme.

La modification logicielle nécessaire

Le sketch v0.8.8 attend le JSON du Shelly et contrôle son identité, sa puissance, sa tension, son courant et sa fréquence. Remplacer son IP par celle de Home Assistant, ou publier un nombre sur MQTT, ne suffit pas. Il faut un lecteur de source adapté, puis conserver le format et les échanges APS validés côté ECU.

L'API REST de Home Assistant peut exposer /api/states/sensor.puissance_reseau avec une authentification Bearer [H2] ; remplacer ce nom d'entité. WebSocket ou MQTT sont d'autres voies à développer pour un suivi continu. Le sketch actuel ne fournit pas d'abonnement MQTT.

Ordre de mise au point

1. Définir la mesure voulue en watts : positive pour le soutirage, négative pour l'export. Additionner deux compteurs seulement si leurs périmètres le justifient : ne pas additionner le compteur général et un sous-compteur déjà inclus.

2. Transmettre la valeur avec sa validité et l'âge réel de l'acquisition. Ne pas transformer unknown, unavailable ou une mesure périmée en 0 W. Une valeur inchangée peut être récente ; last_changed ne suffit pas pour en juger. Prévoir un comportement explicite si la source est perdue.

3. Comparer d'abord la source domotique au Shelly, puis refaire la signature de la page 7 et l'essai physique de la page 8. Valider également la reprise après une coupure de réseau ou de Home Assistant.

Si la domotique est insérée dans le trajet de mesure, son serveur devient nécessaire au fonctionnement de ce trajet. Utilisée uniquement pour le suivi, elle peut être arrêtée sans interrompre la liaison directe Shelly–ESP32.

13 Exploiter le montage et le transmettre

Confirmer que le décalage est revenu à 0 W, puis conserver le sketch adapté, la fiche du site et les deux JSON. L'ESP32 sélectionné reprend la lecture et le répondeur après une reconnexion ou un redémarrage selon son journal. Le PC sert aux essais et à l'analyse ; il n'a pas à rester allumé pour les échanges Shelly/ESP32/ECU. Vérifier la reprise sur le site cible.

Pour restaurer la source réseau initiale, utiliser « Retour aux tores réseau » et attendre la confirmation de l'ECU. Une coupure Wi-Fi ne réalise pas cette restauration. L'intégration du second Shelly pour l'ACC reste à faire, avec la validation décrite page 12.

Situation	Action utile
Compilation : SSID vide	Renseigner WifiLocal.h ou utiliser Preparer_ESP32.cmd avec l'ancien sketch configuré.
Identité inattendue	Comparer les valeurs lues aux constantes du sketch et des deux lecteurs ; corriger les paramètres, puis recompiler si nécessaire.
Réglages différents	Vérifier Grid et PV, CT_GRID_BACKUP et switchCtRoleMatches(). Conserver les contrôles, adapter leurs valeurs au site.
assist déjà présent	Ne pas superposer les collectes. Terminer ou diagnostiquer l'opération précédente ; conserver le relevé partiel.
0 W ou donnée ancienne	Contrôler les dates, la validité de la source et les nouvelles écritures ; ne pas assimiler un cache inchangé à une mesure fraîche.
Pas de réaction à +1 000 W	Revoir la sélection, le mode zéro export et la réserve solaire ; examiner les JSON avant d'attribuer l'échec au protocole.

Périmètre de cette édition publique

Les adresses, identités d'appareils, accès Wi-Fi et relevés du site ont été retirés. Les exemples chiffrés sont théoriques. Les programmes et extractions ne sont pas joints : un éventuel partage de ces fichiers nécessite leur propre nettoyage.

Documentation officielle

S1 Arduino IDE pour Windows macOS et Linux

S2 Installation du support Arduino ESP32

S3 REA et configuration des moteurs

S4 Installation de WSL

S5 Releases officielles Ghidra

S6 Composant EM1 de Shelly

H1 Home Assistant : intégration Shelly

H2 Home Assistant : API REST

H3 Home Assistant : MQTT

Les étapes propres à l'ECU proviennent des sources du prototype, des analyses et des essais réalisés. Les liens officiels documentent les outils ; ils ne certifient pas ce montage. Les autres versions ECU, postes et interfaces domotiques doivent être revalidés.